Appendix A. Basic Usage of R

Data types, packages and functions, permutations, resampling and loops, visualization

Contents

A.1 Data types and operations **2**

A.2 Packages and functions **4**

A.2.1 The pipe operator 4

A.2.2 Writing your own functions 5

A.3 Expressions and functions frequent in nonparametric statistics **6**

A.3.1 Permutations and combinations 6

A.3.2 Random shuffles and bootstrap resampling 7

A.3.3 Repetition 8

A.4 Data visualization **9**

A.5 R code of this book **9**

Overview

The book assumes a reader familiar with basic R. Readers new to R should consult the standard introduction, Venables et al. (2021).[1] This appendix explains the R usage needed for the statistics, resampling and visualization in the book.

A.1 Data types and operations

The simplest way to enter data in R is a vector. E.g. the recovery times $19, 22, 24, 26$ of Example 3.1 in Chapter 3 can be entered as `c(19,22,24,26)`; `c()` combines its inputs into a vector, here stored as an object named `x`:

```
x = c(19,22,24,26)
x
## [1] 19 22 24 26
```

To create a matrix directly use `matrix()`. From a vector `x` of 24 values, a 4×6 matrix:

```
x = c(128, 128, 160, 142, 157, 179,
      122, 137, 177, 177, 160, 138,
      207, 188, 181, 164, 155, 175,
      120, 208, 199, 194, 177, 195)
matrix(x, byrow = TRUE, ncol=6)
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]  128  128  160  142  157  179
## [2,]  122  137  177  177  160  138
## [3,]  207  188  181  164  155  175
## [4,]  120  208  199  194  177  195
```

The argument `byrow = TRUE` fills the matrix by rows (horizontally first). The default is `byrow = FALSE`, so omitting it fills by columns (vertically first):

```
X = matrix(x, nrow = 4)
X
##      [,1] [,2] [,3] [,4] [,5] [,6]
## [1,]  128  157  177  207  155  199
## [2,]  128  179  177  188  175  194
## [3,]  160  122  160  181  120  177
## [4,]  142  137  138  164  208  195
```

Data for statistical analysis are stored as data frames; e.g. reading the file `worldbank2020.csv` gives a data frame:

```
wbdata = read.csv("data/worldbank2020.csv")
str(wbdata)
## 'data.frame':   153 obs. of  9 variables:
##  $ Overall.rank          : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ Country.or.region     : chr  "Finland" "Denmark" ...
##  $ Score                 : num  7.81 7.65 7.56 7.5 7.49 ...
##  $ GDP.per.capita        : num  1.28 1.33 1.39 1.33 1.42 ...
##  $ Social.support        : num  1.5 1.5 1.47 1.55 1.5 ...
##  ...
```

`wbdata` stores 9 vectors of equal length as columns, each vector holding the observations of one variable. To extract the third variable `Score` of `wbdata` use `wbdata$Score` or `wbdata[,3]`. With `with()` the variable names inside a data frame can be called directly:

[1] Venables et al. (2021) is freely available at https://cran.r-project.org/doc/manuals/r-release/R-intro.pdf.

```
with(wbdata, lm(Score ~ GDP.per.capita))
##
## Call:
## lm(formula = Score ~ GDP.per.capita)
##
## Coefficients:
##    (Intercept)  GDP.per.capita
##          3.462           2.316
```

R operations are elementwise by default; e.g. the sum and product of two vectors are elementwise:

```
x = c(1,2,3,4)
y = c(5,6,7,8)
x + y
## [1]  6  8 10 12
x * y
## [1]  5 12 21 32
```

The product (`*`) of two matrices is also elementwise:

```
(X = matrix(1:6,nrow =3, ncol = 2))
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
X * X
##      [,1] [,2]
## [1,]    1   16
## [2,]    4   25
## [3,]    9   36
```

Sums and differences of matrices are likewise elementwise; the results of `X + X` and `X - X` are easy to guess. For the matrix product use `%*%`:

```
(Y = matrix(1:6, nrow = 2, ncol = 3))
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
X %*% Y
##      [,1] [,2] [,3]
## [1,]    9   19   29
## [2,]   12   26   40
## [3,]   15   33   51
```

The transpose of a matrix uses `t()`: X' is `t(X)`. The inverse uses `solve()`: if X^{-1} exists it is `solve(X)`. With these, e.g., the regression hat matrix $X(X'X)^{-1}X'$ is computed:

```
(H = X %*% solve(t(X) %*% X) %*% t(X))
##            [,1]      [,2]       [,3]
## [1,]  0.8333333 0.3333333 -0.1666667
## [2,]  0.3333333 0.3333333  0.3333333
## [3,] -0.1666667 0.3333333  0.8333333
```

What if a vector y of length 3 is multiplied by the 3×3 matrix H? Viewing y as a 1×3 matrix (row vector) the product yH is defined; viewing it as 3×1 (column vector), Hy is defined. R turns the vector y into a row or column vector automatically so that the product is defined:

```
y = c(0,1,2)
H %*% y # H times y
```

```
##              [,1]
## [1,] 4.440892e-16
## [2,] 1.000000e+00
## [3,] 2.000000e+00
y %*% H # y times H
##               [,1] [,2] [,3]
## [1,] -4.440892e-16    1    2
```

(Supplied remark: $y = (0, 1, 2)'$ lies in the column space of X, here $y = X(-\frac{2}{3}, \frac{1}{3})'$; so $Hy = y$ up to rounding error 4×10^{-16}.)

A.2 Packages and functions

An R package is a bundle of objects such as functions and data sets. The functions `matrix()`, `t()`, `solve()` above are in R's `base` package; `read.csv()` is in R's `utils` package. These two, together with packages such as `graphics` and `datasets`, are loaded automatically in an R session. To use functions and data sets of a package not included in R, run `library(package name)`. E.g. to use functions of the `tidyverse` package, run the following to "load" `tidyverse` into the R session:

```
library(tidyverse)
```

When using the package for the first time, install it with `install.packages("tidyverse")`. (Installation is needed only once, but loading must be done every time an R session starts.) `tidyverse` is in fact a collection of several packages; e.g. `ggplot2` and `dplyr` are included in it.

With many packages, the same function or data set name may be defined in two packages. Then `package::function()` or `package::dataset` specifies exactly which object of which package is meant. (If the package is installed, this calls the object directly without loading via `library()`.)

Below, the first function `select` belongs to the package `dplyr` (already loaded in the R environment by `library(tidyverse)`):

```
head( select(wbdata, Score) )
##   Score
## 1 7.809
## 2 7.646
## 3 7.560
## 4 7.504
## 5 7.488
## 6 7.449
```

`select(wbdata, Score)` selects only the variable `Score` of the data set `wbdata`. To use the `select` function of the `MASS` package, use `MASS::select()`. We have not loaded the `MASS` package, but `::` makes its functions usable:

```
MASS::select(MASS::lm.ridge(Score ~ Generosity + Social.support,
                            data = wbdata, lambda = seq(0,1,0.001)))
## modified HKB estimator is 0
## modified L-W estimator is 0
## smallest value of GCV  at 1
```

A.2.1 The pipe operator

Loading `dplyr` makes the special operator `%>%` available. The pipe operator `%>%` makes R code easier to read. E.g. for the mean (`mean()`) of a vector `x` one usually writes

```
mean(x)
## [1] 2.5
```

but in reading order one can write

```
x %>% mean
## [1] 2.5
```

The value before the pipe (vector, matrix, data frame) becomes the first input of the function `mean()` after the pipe. With the pipe, `head( select(wbdata, Score) )` becomes:

```
wbdata %>% select(Score) %>% head()
```

If the last function has no other inputs, the parentheses may be omitted.

A.2.2 Writing your own functions

To repeat the same task, it is often useful to define functions. E.g. to rank-transform observations `x` within each block `block`:

```
x = (1:6)^2;block = rep(c("a","b"), each = 3)
(temp.data = data.frame(x,block))
##    x block
## 1  1     a
## 2  4     a
## 3  9     a
## 4 16     b
## 5 25     b
## 6 36     b
out = temp.data %>% dplyr::group_by(block) %>% dplyr::mutate(ranks = rank(x))
out$ranks
## [1] 1 2 3 1 2 3
```

For blockwise ranking we used `group_by()` and `mutate()` of `dplyr`: the observations of `temp.data` are grouped by `block` (`group_by()`), and a new variable `ranks` is created by transforming the existing variable `x` within each group (`mutate()`). To repeat this for several data sets, define a function named `rankinblock`:[2]

```
rankinblock = function(x,block){
  require(dplyr)
  out = data.frame(x,block) %>% group_by(block) %>% mutate(ranks = rank(x))
  return(out$ranks)
}
```

`require(package)` plays the same role as `library(package)` but is designed for use inside functions; see `?require`. The defined function is used as:

```
rankinblock(x,block)
## [1] 1 2 3 1 2 3
```

[2] This function is in `np_supp.R`.

A.3 Expressions and functions frequent in nonparametric statistics

A.3.1 Permutations and combinations

Reordering all objects of a sequence `1:n` or vector `v` of length n is a *permutation*; in this book "shuffle" is used instead of "permutation." The number of all permutations of n objects is

$$n! = n(n-1)\cdots 2\cdot 1,$$

computed in R by

```
n = 10
factorial(n)
## [1] 3628800
```

Choosing r of the n elements of a vector `v` and arranging them is also a permutation; the number of such shuffles is $P(n,r) = n!/(n-r)! = n(n-1)\cdots(n-r+1)$. `permutations()` of the `gtools` package generates all of them (install with `install.packages("gtools")` if needed). The code below generates all shuffles choosing and ordering $r = 2$ objects from `v = 1:4` of length $n = 4$:

```
gtools::permutations(n = 4, r = 2, v = 1:4)
##       [,1] [,2]
##  [1,]    1    2
##  [2,]    1    3
##  [3,]    1    4
##  [4,]    2    1
##  [5,]    2    3
##  [6,]    2    4
##  [7,]    3    1
##  [8,]    3    2
##  [9,]    3    4
## [10,]    4    1
## [11,]    4    2
## [12,]    4    3
```

The exact permutation tests of Part I mainly use combinations rather than permutations. Choosing r objects from a vector `v` of length n without regard to order is a combination; their number is

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

($\binom{n}{r}$ is read "n choose r"), computed in R by

```
choose(4,2)
## [1] 6
```

The code below generates all shuffles choosing $r = 2$ objects from `v = 1:4` of length $n = 4$ without regard to order:

```
gtools::combinations(n = 4, r = 2, v = 1:4)
##      [,1] [,2]
## [1,]    1    2
## [2,]    1    3
## [3,]    1    4
## [4,]    2    3
## [5,]    2    4
## [6,]    3    4
```

Comparing two populations, suppose we have n_1 observations `x1` from the first and n_2 observations `x2` from the second. If the two populations were in fact one, the n_1 observations `x1*` chosen at random from the pooled sample `c(x1,x2)` could have been the sample from the first population (see Chapter 3). To count all such cases, the number of combinations (not permutations) is needed; combinations too can be enumerated by shuffling the pooled sample, hence the term "shuffle."

```
set.seed(1)
x1 = rpois(n = 3, lambda = 5) # n1 = 3
x2 = rpois(n = 2, lambda = 5) # n2 = 2
(x.combined = c(x1,x2))       # n = 5
## [1] 4 4 5 8 3
```

The code below generates all shuffles choosing $r = 3$ objects from `x.combined` of length $n = 5$ without regard to order:

```
gtools::combinations(n = 5, r = 3, v = x.combined, set = F)
##       [,1] [,2] [,3]
##  [1,]    4    4    5
##  [2,]    4    4    8
##  [3,]    4    4    3
##  [4,]    4    5    8
##  [5,]    4    5    3
##  [6,]    4    8    3
##  [7,]    4    5    8
##  [8,]    4    5    3
##  [9,]    4    8    3
## [10,]    5    8    3
```

The vector `x.combined` contains the value 4 twice. In `gtools::combinations()`, the argument `set = F` forces the duplicate values of the input not to be removed. Also, `set.seed(1)` was used so that random sampling from the Poisson distribution (`rpois()`) always gives the same result. For reproducible analyses, always call `set.seed()` with a fixed number before random sampling.

A.3.2 Random shuffles and bootstrap resampling

Considering all shuffles is possible only for small sample sizes n. Even a not-large $n = 30$, choosing $n_1 = 15$, gives $\binom{30}{15} = 155117520$ (about 155 million) shuffles. So it is usual to sample at random only M of the $\binom{n}{n_1}$ possible shuffles, for a predetermined $M = 1999$ or $M = 9999$, to "approximate" all possible shuffles. Random sampling uses `sample`. The code below draws 3 elements of `x.combined` by simple sampling without replacement:

```
set.seed(1)
sample(x.combined, size = 3)
## [1] 4 8 5
sample(x.combined, size = 3)
## [1] 4 4 3
```

Being random sampling, the result differs every run. The next code adds the argument `replace = FALSE` to emphasize sampling without replacement; the default of `replace` in `sample()` is `FALSE`, so the result is the same as without it:

```
set.seed(1)
sample(x.combined, size = 3, replace = FALSE)
## [1] 4 8 5
```

The next code draws all 5 elements of `x.combined` of length 5 without replacement, the same as shuffling the 5 observations at random:

```
sample(x.combined, size = 5)
## [1] 5 4 8 4 3
```

Bootstrap resampling in Part II uses simple sampling with replacement of n objects from a sample `x.combined` of length n; set `replace = TRUE`:

```
sample(x.combined, size = 5, replace = TRUE)
## [1] 4 4 3 5 4
```

A.3.3 Repetition

Random permutation tests and bootstrap resampling repeat simple sampling with or without replacement many times. The most basic way to repeat is the control statement `for`. The code below repeats $M = 1999$ times the computation of the mean of 50 objects drawn without replacement from the variable `Score` of the data frame `wbdata`:

```
M = 1999
set.seed(2)
rand.mean = vector(length = M)
for(i in 1:M){
  rand.mean[i] = sample(wbdata$Score, size = 50) %>% mean
}
head(rand.mean)
## [1] 5.45034 5.42934 5.59312 5.60280 5.62102 5.53988
```

With `replicate()` this can be written shorter and more intuitively:

```
set.seed(2)
rand.mean = replicate(M, sample(wbdata$Score, size = 50) %>% mean)
head(rand.mean)
## [1] 5.45034 5.42934 5.59312 5.60280 5.62102 5.53988
```

`replicate()` is mostly used for repeated random sampling.

Repeated computation over each row (or each column) of a matrix is also needed. As an example take the 10×3 matrix `allperms`:

```
allperms = gtools::combinations(n = 5, r = 3, v = x.combined, set = F)
```

To compute the standard deviation of the values of each row of `allperms`, the most basic way is the control statement `for`:

```
sd.allperms = vector(length = choose(5,3))
for(i in 1:choose(5,3)){
  sd.allperms[i] = allperms[i,] %>% sd()
}
round(sd.allperms,2)
## [1] 0.58 2.31 0.58 2.08 1.00 2.65 2.08 1.00 2.65 2.52
```

Less intuitive but faster than a `for` statement is `apply()`. Reading the code below: for each row (the 1 means rows, 2 would mean columns) of the matrix `allperms`, compute the standard deviation `sd()` and round to two decimals:

```
apply(allperms, 1, sd) %>% round(2)
## [1] 0.58 2.31 0.58 2.08 1.00 2.65 2.08 1.00 2.65 2.52
```

Now compute the coefficient of variation (standard deviation / mean) of each column of `allperms`. The function computing the coefficient of variation and rounding to two decimals is given directly as the third argument of `apply()`:

```
apply(allperms, 2,
      function(x){
        cv = sd(x)/mean(x)
        round(cv,2)
      })
## [1] 0.08 0.31 0.50
```

A.4 Data visualization

The best way to understand data and their distribution is suitable visualization. Most figures of the book were produced with `ggplot()` of the `ggplot2` package included in `tidyverse`. A few examples follow; to learn more see Wickham and Grolemund (2016).[3]

Univariate data. The most representative way to visualize the distribution of a variable from its observations is the histogram:

```
x = rnorm(100, mean = 1, sd = 1)
data.frame(x = x) %>%
  ggplot(aes(x = x)) +
  geom_histogram(aes(y = ..density..), binwidth = .5,
                 fill = "grey", color = "white") +
  geom_rug()
```

To see whether the distribution of the data `x` follows the parametric $N(1, 1)$, overlay the density of $N(1, 1)$ on the histogram, or produce a quantile plot (qqplot). Producing a QQ envelope plot, a quantile plot with confidence bands, with `car::qqPlot()` is recommended.

```
data.frame(x = x) %>%
  ggplot(aes(x = x)) +
  geom_histogram(aes(y = ..density..), binwidth = .5,
                 fill = "grey", color = "white") +
  geom_function(fun = dnorm, args = list(mean = 1, sd = 1))
car::qqPlot(x, dist = "norm", mean = 1, sd = 1)
```

To compare the distributions of grouped univariate data by group, use box plots:

```
farm = read.csv("data/smartfarm.csv")
farm %>% ggplot(aes(x = block, y = x, color = treatment)) +
  geom_boxplot()
```

Bivariate data. For visualizing bivariate data use a scatter plot:

```
wbdata %>% ggplot(aes(x = GDP.per.capita, y = Score)) +
  geom_point()
```

(The resulting histogram, density overlay, QQ envelope plot, box plots and scatter plot are Figures D.1–D.5 of the book.)

A.5 R code of this book

All computations and visualizations of the book can be found at https://github.com/sungkyujung/npbook. The functions defined to simplify the discussion are collected in `np_supp.R`; to use them,

[3] https://r4ds.had.co.nz/

run

```
source("np_supp.R")
```